

THE ONE

SOFTWARE DEVELOPMENT PACK – PROTOTYPE → PRODUCTION



- 1 Executive summary
- 2 Product overview & principles
- 3 Personas & roles
- 4 Operational SOP – the work-order lifecycle
- 5 User journeys
- 6 Functional scope & requirements
- 7 Architecture, data model & API
- 8 Development SOP (how we build)
- 9 QA, acceptance & release gates
- 10 Release plan & current position
- 11 Risks, open decisions & security notes
- 12 Document map

Version 1.0 · 2026-08-07 · Prepared for Jordan Brown – Seamless FM / Byblos Vista Consulting

Status: **Draft for review.** This pack consolidates and extends the repository's authoritative documents; where it summarizes, the source file is cited and remains the source of truth.

1 EXECUTIVE SUMMARY

The One (working name) is a purpose-built operations platform that replaces the ClickUp workspace currently running **Seamless FM / Byblos Vista / Alpha Fixers** — one facilities-maintenance operation trading under multiple brands, with ~98 workspace users and ~21,000 historical work orders. Work orders arrive from client CMMS platforms (Corrigo, Ecotrak, ServiceChannel), email, and the company's own Seamgo tool; they are accepted, dispatched to external subcontractor technicians, quoted, approved by the client, fulfilled, and then paid on both sides — the technician (AP) and the client invoice (AR).

A working prototype exists: a local monorepo app (React + Fastify + embedded Postgres) covering Sprints 1-5 — work-order table and record, 19-status pipeline with phase bar, quote builder with role-gated approval, technician payment requests, an AR audit/invoicing queue, and an obligation ("Pulse") engine — all seeded with real ClickUp data, including hero work order **WO-39403**.

What this pack is. The repository already contains strong primary documents (process map, roadmap, architecture proposal, sprint spec). What it lacked was the packaged layer a development effort is handed: named personas, written user journeys, a development SOP, a QA/release process, and a consolidated risk register. This document supplies those, and indexes the primary documents rather than duplicating them (§12).

Current position at a glance

Dimension	State
Release stage	R0 Prototype (Sprints 1-4) complete and committed; Sprint 5 (obligations, Pulse, Receivables) built but uncommitted
Working software	7 live screens, 30+ API endpoints, 4 SQL migrations, idempotent seed of 28 real work orders
Next gate	R1 Pilot (~5 users): dispatch runs the "Incoming WOs" intake for one week with zero ClickUp fallback
Biggest process gaps	Single git commit (no restore points); repo lives in OneDrive without a remote; 9 open process questions in the lifecycle doc; Corrigo/ServiceChannel API feasibility unproven

2 PRODUCT OVERVIEW & PRINCIPLES

The problem

Operations run today on ClickUp plus a constellation of side tools: **Yoda** (quote builder), **PPR** (tech payment requests), **Argus** (AR completion-audit assistant), **Quotato** (Quo/OpenPhone call & SMS capture with AI quote drafting), an **n8n escalation bot** watching the contact inbox, and a Teams PDF side-channel for assignments. ClickUp provides primitives (custom fields, statuses, lists, views) but no lifecycle awareness, no money model, no vendor module, and no client-sync boundary. The One rebuilds the primitives ClickUp actually uses — measured from the live workspace: 19 statuses, 102 custom fields, 163 lists — and makes the work-order lifecycle native, so the side tools can be absorbed rather than maintained separately.

Product principles

Source: `product/feature-roadmap.md (v1)` — abbreviated.

Principle	Meaning
1 · API-first system of record	n8n, Supabase, HubSpot, QuickBooks, OpenPhone, Power BI are the constellation; The One is the sun. Every entity readable/writable via API + webhooks from day one.
2 · Primitives, not features	Custom fields, statuses, views, automations are configuration. Work orders, audits, vendor records are configurations of primitives.
3 · Lifecycle-native	The 15 stages of the WO lifecycle (§4) are the spine of the product, not an afterthought.
4 · Prototype first; the schema is the contract	v0 is local and minimal; what must be right early is the field engine, membership model, <code>client_visible</code> , vendor + payable entities, and the activity log.
5 · Beat Facilio where it's weak	Live SQL access for Power BI, escalation logic designed for external vendors, integration adapters as first-class modules. Facilio (~\$25K/yr) remains the evaluated fallback — leverage, not Plan A.

What exists in the prototype today

Screen	Purpose
Work Orders (/)	KPI row, status-group tabs (All/Open/Active/Done/Closed), search, obligation clock chips per row, sort-by-breach.
WO Detail	Tabs: Overview · Messages · Audit trail · All fields. Phase bar, updates feed with <code>client_visible</code> toggle, photos, soft-close checklist, money card + NTE meter, payables, people/site/dates, obligations rail.
Quote Builder	The Yoda replacement: INCURRED + PROPOSED option sections, line items with drag-reorder, live money rail (cost / grand total / tax / NTE / profit / margin), auto-generated client summary, role-gated Submit and Approve & Send (locked, never hidden).
Request Payment	The PPR replacement: payee (vendor or manual), purpose, method, alternate recipient, prior payments on the WO.
Quotes index	All quotes: WO, status, grand total, last touched.

Pulse	Triage columns: Needs me now (tier 2–3) · Due soon (tier 1) · Watching (tier 0); snooze requires a reason.
Receivables	Audit tab (Grey-Flag completion-audit queue, ported from Argus rules) and Invoicing tab (Ready → Sent → Paid).

Navigation also shows **Dashboard, Vendors, Admin** as visible-but-disabled ("Coming in a later sprint") so the product's eventual shape is legible, and an actor switcher ("Viewing as") stands in for auth until R1.

3 PERSONAS & ROLES

Source: `product/wo-lifecycle.md` roles glossary; `app/packages/shared/src/index.ts` role enum; seed personas.

Internal personas

Code	Role	Responsibilities & authority	Prototype role
TL	Team Lead	Accept/decline authority; audits quotes; can push to client CMMS.	<code>tl</code>
ATL	Assistant Team Lead	Same touchpoints as TL; the approve-and-send gate on quotes.	<code>atl</code>
OA	Operations Admin	Intake into the system; performs the "TL Rep" push of curated updates to the client's CMMS.	—(R1)
AM	Account Manager	Accept/decline authority; owns the client relationship.	<code>am</code>
OM	Operations Manager (dispatcher)	Sources technicians, dispatches both visits, soft-closes. Tiers: Ops Coordinator (probation) → OM → Senior OM (quote-builder access starts here).	<code>om</code> / <code>senior_om</code>
HAR / AR	Accounts Receivable	AR audit, invoicing, collections. Tiers: AR (probation) → AR → Senior AR → HAR.	—(R1)
AP	Accounts Payable	Pays technicians at final soft close; orders parts (<code>ap will order</code> / <code>ap ordered</code>).	—(R1)
Exec guest	Executive read-only	Rollup dashboards only (e.g. Teresa, Rocio).	—(R2)
Admin	Platform admin	Full access; default seeded actor.	<code>admin</code>
Service accounts	Bots / n8n	First-class principals; every automated write is attributed like a human's.	seeded

Trust tiers are a hard requirement, not a nice-to-have. OM and AR both run probation → standard → senior ladders; permissions and review queues must be tier-aware (RBAC lands in R1, S5–S6).

External personas (not platform users)

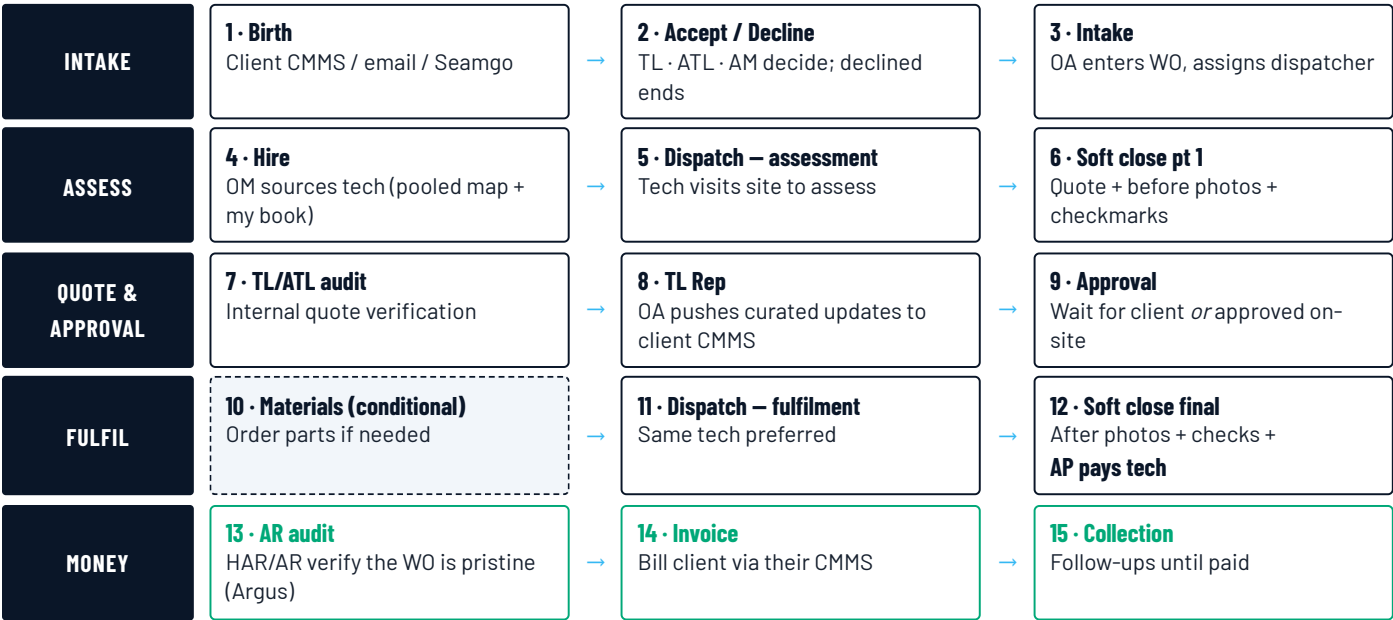
Persona	Relationship
Technician	External subcontractor, sourced per job from a pooled vendor map + per-OM "my book". Reached by SMS/call (Quo/OpenPhone); never logs in before the R3 vendor portal. Paying techs fast is a vendor-retention lever.
Client / FM company	Issues WOs from their CMMS (Corrigo, Ecotrak, ServiceChannel), approves quotes there, receives only <code>client_visible</code> updates — never the full internal picture.

Demo personas (seeded, for UAT)

Jordan Brown (`admin` , default actor) · Elise (`at1` — approves & sends) · Matt Hammond (`senior_om` — dispatcher of hero WO-39403) · Peter Hope (`am`) · Zach Malden (`t1`) · all other seeded members default to `om` , deliberately below the quote gate so the 403 path is testable.

4 OPERATIONAL SOP – THE WORK-ORDER LIFECYCLE

Source of truth: `product/wo-lifecycle.md` (v3, 2026-07-27). The One must support every stage at parity before improving on it. Summary below; the source file carries the full narrative and its 9 open questions (§11).



Operating rules embedded in the lifecycle

- Linear with branches, not a loop. Canonical shape is two visits — assess, then fulfil. On-site approval skips the wait; materials is conditional: `return_trip_needed` is the exception path.
- Selective outbound sync. The client's CMMS receives edits, status changes, and curated messages — never the full internal picture. Every update carries an internal-vs-client-facing flag (`client_visible`).
- Money flows both ways on the WO. AR bills clients (revenue); AP pays technicians at soft close (cost). Profit-per-WO is the two nets. Money card: NTE · quote · cost · invoiced · profit · margin.
- The `!! waiting for approval` bottleneck is client-side (46 WOs in the sample). The One tracks aging and prompts follow-ups; it cannot eliminate the wait.

Status pipeline

One 19-status pipeline everywhere (no per-list overrides), grouped **open · active · done · closed**, rolled up into 9 phases for the phase bar: **Intake · Assessment · Quote · Approval · Scheduled · In Progress · Parts · Done · Invoiced** (Parts conditional; `!! canceled/postponed` is off-pipeline; archive status `invoiced`).

Lifecycle stage	ClickUp statuses (mapping to confirm)
Birth / Accept	<code>Open</code> , <code>emergency</code> (declined WOs never enter)
Intake → Hire → assessment	<code>assessment scheduled</code> , <code>assessment ongoing</code> , <code>return trip needed</code>
Soft close pt 1 / quote	<code>waiting for quote</code> , <code>quote ready</code>
TL Rep → client decision	<code>!! waiting for approval</code> , <code>!! waiting for advice</code>

Approved → materials → fulfilment	approved, please order parts, waiting for parts, job scheduled, job ongoing, pm scheduled
Final soft close → AR	done/incurred, !! ready to invoice
Invoice / Collection	<< invoiced not paid >> → archive invoiced
Off-ramp	!! canceled/postponed

5 USER JOURNEYS

Seven journeys cover the platform end to end. Each is written against the as-is process (§4) and annotated with what the prototype already implements **BUILT** versus what lands in a later release **R1+**.

J1 · Intake & acceptance

OA · TL · ATL · AM

Trigger: a new WO appears on a client CMMS, in email, or in Seamgo. **Outcome:** an accepted WO sits in a dispatcher's book, or a decline is recorded.

1. OA monitors the intake sources and surfaces the new WO (today: manual re-entry into ClickUp — the automation win is inbound adapters, Ecotrak first). **R1-R2**
2. TL / ATL / AM decides accept or decline; a decline ends the flow with a reason. **R1 structured**
3. OA creates the record — ordered intake form (the numbered 13.–38. fields), client, site, trade, NTE, SLA date. **BUILT** (record + fields; creation UI is R1)
4. The WO is routed to an OM's book — assignment, not list-moving. Emergency WOs enter as `emergency` and start a 2-hour acknowledgment clock. **BUILT** (obligation rule `emergency_ack`)

Pain removed: multi-source inbox aggregation; the Teams PDF dual-assignment channel is retired at R1-R2.

J2 · Sourcing & assessment dispatch

OM (DISPATCHER)

Trigger: a WO lands in the OM's book. **Outcome:** a technician has assessed the site and the OM holds photos + findings.

1. OM searches the pooled vendor map (trade × location) alongside their own book; gaps are filled from online directories. **R1 · S6**
2. OM contacts the tech by SMS/call; the Quo thread (calls, transcripts, AI summaries, MMS photos) attaches to the WO's Messages tab. **BUILT** (mirrored data; live OpenPhone sync via Quotato integration later)
3. OM schedules the visit: status → `assessment scheduled` → `assessment ongoing`. A schedule-owed clock (2 business hours after approval-type triggers) keeps dispatch honest. **BUILT**
4. Tech reports back; before photos and findings land on the WO. **BUILT** (updates feed + photos)

Exception: jobs approved up front skip straight to fulfilment (J4).

J3 · Quote build → approval → send

SENIOR OM → ATL/TL → OA

Trigger: assessment findings are in; status `waiting for quote` and a 2-business-day quote-owed clock is running.

Outcome: the client's CMMS holds the quote; status `!! waiting for approval`.

1. Senior OM opens the Quote Builder (roles below `senior_om` see the controls locked with a tooltip — never hidden). **BUILT**
2. Builds INCURRED (work already performed — context, not billed into the total) and one or more PROPOSED options with narrative and line items (service / labor / part / material; `amount = qty × rate × 1.5 if OT`, computed never stored). Live rail shows cost, grand total, tax (by state), NTE meter, profit, margin. **BUILT**
3. Submit → `pending_approval`. A 4-business-hour quote-review clock now sits on the ATL. **BUILT**

4. ATL/TL audits and either rejects (back to draft, with feedback) or approves & sends → `sent`. **BUILT**
5. OA performs TL Rep: pushes the auto-generated client summary + status to the client's CMMS (only `client_visible` content). **R2 adapter; manual today**
6. Waiting on the client, a 5-business-day approval-followup clock prompts chasing. On-site approvals skip the wait entirely. **BUILT**

J4 · Parts & fulfilment

OM · AP

Trigger: client approval (or approved on-site). **Outcome:** approved work performed on site.

1. If materials are needed: `please order parts` → AP orders (`ap will order` / `ap ordered` tags today; structured parts stage at R1) → `waiting for parts`. **R1 · S7+**
2. OM redispaches — same tech preferred: `job scheduled` → `job ongoing`. **BUILT** (status flow)
3. Extra visits when needed (`return trip needed`) — the exception, not the shape.

J5 · Final soft close & technician payment

OM · AP

Trigger: work completed on site. **Outcome:** WO documentation complete; the tech's payable is settled.

1. OM uploads after photos and completes the soft-close checklist. **BUILT**
2. OM (or AP) raises a payment request: payee = vendor record or manual name+phone, purpose, amount (hardened parsing), method (Zelle, ACH, check, company card, Cash App), optional alternate recipient; prior payments on the WO are shown to prevent doubles. **BUILT**
3. The request moves `requested` → `approved` → `paid` in the AP queue (the control point — creation is deliberately ungated); a 2-business-day payment-processing clock sits on AP. **BUILT** (model + clock; AP queue UI at R1)
4. Status → `done/incurred`; the WO's cost side is now real, so profit-per-WO is real.

Why it matters: paying techs fast and reliably is a vendor-retention lever — good techs go where they get paid.

J6 · AR audit → invoice → collection

HAR · SENIOR AR · AR

Trigger: WO reaches final soft close. **Outcome:** cash collected; WO archived `invoiced`.

1. The WO enters the Receivables **Audit** queue. Grey-Flag rules (ported from Argus) check the record automatically: check-in/check-out present, client quote attached, SharePoint deliverables, CO# format, Admin/Quote release gate. **BUILT**
2. AR reviewer clears flags or bounces the WO back to the dispatcher to fix. Trust tiers decide who may release. **R1 RBAC**
3. Clean WOs move to **Invoicing**: Ready → Sent (entered into the client's CMMS) → Paid. **BUILT**
4. Collections works `<< invoiced not paid >>` with calls and follow-ups until payment; aging feeds the KPI row and (R2) a collections queue with promise-to-pay. **R2**

Trigger: continuous — the obligation engine derives clocks from WO/quote/payment state (no cron; a debounced evaluator runs on reads and after relevant writes). **Outcome:** nothing owed goes silent.

1. User opens **Pulse**: *Needs me now* (tier 2 breached / tier 3 critical) · *Due soon* (tier 1, $\geq 80\%$ of clock) · *Watching* (tier 0 ambient). **BUILT**
2. Clocks respect business hours (Mon–Fri 08:00–18:00 CT, paused outside) except emergencies, which run 24×7. Each obligation pings once per tier — ever. **BUILT**
3. The user either **resolves by evidence** — do the owed thing (send the quote, schedule the job, pay the tech) and the obligation clears itself — or **snoozes** with a mandatory reason, max 72h. There is deliberately no dismiss. **BUILT**
4. The bell inbox mirrors pings; read/read-all supported. **BUILT**

V1 rules (config-as-data): `emergency_ack` 2h 24×7 · `quote_owed` 2 bus. days · `schedule_owed` 2 bus. hours · `approval_followup` 5 bus. days · `quote_review_owed` 4 bus. hours (ATL) · `payment_processing` 2 bus. days (AP) · `sla_blown` on SLA date passed.

6 FUNCTIONAL SCOPE & REQUIREMENTS

Source of truth: *product/feature-roadmap.md* (12-layer feature map, every item tagged); *platform-brainstorm/clickup-feature-inventory.md* (tiered T0–T3 requirements catalogue). Condensed view:

Layer	Scope	Lands
L0 · Foundation	Custom-field engine (13 types), 19-status pipeline + phases, WO record, task↔list M2M with home list, attributed activity log, <code>client_visible</code> , attachments, service-account principals, outbox/event stream	P → R1
L1 · Identity & access	RBAC across all roles, trust-tier gating, per-user data scoping, outbound-push gate by role, exportable audit-log UI	R1 → R2
L2 · Portfolio	Clients + FM companies, billing entities (SFM/AF/BKR/TPM/RF/EDS), sites with geo, multi-entity, assets	P → R3
L3 · Lifecycle engine	Intake queue, accept/decline with reason, routing to books, approval fork, soft-close checklists, quote-audit gate, parts stage, NTE guardrails, SLA aging, exception paths, time-in-status metrics	P → R2
L4 · Vendor management	Vendor DB (trades, geo, rates), pooled map + my book, tiering, compliance docs w/ expiry gating, onboarding, scorecards, invoice capture, 1099, portal, auto-assignment	R1 → R3
L5 · Money	Financial tab, quote builder, approval chains, client invoicing, AP payables, AR audit (Argus absorb), collections queue, QuickBooks sync, rate cards	P → R3
L6 · Integrations	REST API + webhooks (n8n-friendly), Ecotrak adapter first, email-to-WO, Corrigo/ServiceChannel (pending spike), selective outbound sync, Seamgo absorb-or-bridge, client portal	R1 → R3
L7 · Views & analytics	Table + filters, saved/shared views, board, role dashboards with drill-down, approval-aging dashboard, map view, exec dashboards, PBI read-replica, calendar/workload	P → R2
L8 · Communication	Notifications inbox, email digests, VoIP click-to-call + auto call-logging, Teams retirement, vendor SMS	R1 → R3
L9 · Automations	Rules engine (trigger→condition→action, run log, loop guards), SLA escalations, AM auto-assign; n8n external from day one	R1 → R2
L10 · AI	Intake triage (parse WO PDFs/emails), copilot Q&A, vendor validation, outbound vendor calls	R3
L11 · PM & assets	PM schedules, asset-linked recurring WOs, service history	R3

Decided requirements worth restating

- **Quotes:** options are internal (client is not choosing between them); OT multiplies rate ×1.5; sales tax derives from state; the client Summary is auto-generated but text-editable. Build = Senior OM+; approve & send = ATL+.
- **UX standard (from the ui-ux-pro-max audit):** labels above fields; required indicators; disabled ≠ enabled styling; adequate contrast; role-gated actions render locked, not hidden; tabular numerals for money.

- **Obligations:** resolution is evidence-only – no dismiss endpoint exists, by design; snooze is capped at 72h with a mandatory reason; one ping per (obligation, tier) enforced by a DB unique constraint.

7 ARCHITECTURE, DATA MODEL & API

Source of truth: *platform-brainstorm/platform-architecture-proposal.md* (three scored candidates); *app/docs/SPRINT1-SPEC.md* (stack decisions).
Summary:

Architecture decision

Three candidates were scored concern-by-concern: **A** server-authoritative Postgres (≈92), **B** Zero local-first sync (≈83), **C** hybrid relational + CRDT islands (≈80). The recommendation is a **sequence, not a pick**: build A now; adopt Zero at the Phase-2 gate if realtime collaboration demands it; add CRDT islands when collaborative editing (Zeus) lands.

Prototype stack (as built)

Layer	Choice	Rationale / upgrade path
Monorepo	npm workspaces — <code>apps/api</code> , <code>apps/web</code> , <code>packages/db</code> , <code>packages/shared</code>	No build step for server code; <code>tsx</code> runs TS directly
Database	PGlite (embedded Postgres 16 WASM), file-persisted	No Docker/cloud for v0; upgrade = connection-string swap to the committed <code>docker-compose.yml</code> Postgres
DB access	Plain SQL via thin <code>query()</code> ; raw ordered <code>.sql</code> migrations	Migrations append-only, executed whole, numbered 0001-0004
API	Fastify 5 + Zod 3, TypeScript ESM, port 5174	Zod at every boundary; single error envelope
Web	React 18 + Vite 5, TanStack Query 5, React Router 6, port 5173	<code>/api</code> proxied by Vite — no CORS
Styling	Plain CSS custom-property tokens; light/dark via <code>data-theme</code>	No Tailwind / component library; brand kit in <code>UI Essentials/</code> is authoritative
Shared types	<code>@theone/shared</code> — types only	Web never imports the DB runtime
Deferred deliberately	Outbox/event stream, job worker, WS/SSE realtime, full-text search, ACL enforcement, formula engine, automations, ORM	

Data model — what must not be gotten wrong

Entity group	Notes
Hierarchy & principals	<code>container</code> (workspace→space→folder→list) · <code>principal</code> (humans + service accounts, both first-class)
Field engine	<code>status_set</code> / <code>status</code> (groups denormalised onto task) · <code>field_def</code> (13 types; 102 real defs seeded) · promoted hot columns + <code>fields</code> JSONB with GIN index
The WO	<code>task</code> (flat; nullable parent for future subtasks) · <code>task_list_membership</code> M2M with one home list — routing = updating membership , not moving rows

Audit & comms	<code>activity_log</code> append-only, every write attributed · <code>comment.client_visible</code> – the internal-vs-client boundary · Quo tables (<code>quo_conversation/call/message/job_segment</code>) have no <code>client_visible</code> column anywhere: that channel is never client-facing, by schema
Money	<code>vendor</code> · <code>payable</code> · <code>quote</code> (one per task, revisions) / <code>quote_section</code> / <code>quote_line</code> (amounts computed, never stored) · <code>payment_request</code>
Obligations	<code>obligation_rule</code> (config-as-data) · <code>obligation</code> (one live per rule+subject) · <code>obligation_ping</code> (unique per tier) · <code>notification</code>

API conventions (already enforced – keep them)

- Everything under `/api`; Zod-validated inputs; errors as `{ error: { code, message, details } }` with `BAD_REQUEST` | `FORBIDDEN` | `NOT_FOUND` | `INTERNAL`.
- Acting principal resolved once per request from `X-Actor-Id` (pre-auth stand-in; replaced by real auth in R1). Role gates return 403; the UI renders the same gate as a locked control.
- Work orders addressable by UUID or WO number. List endpoints paginate (`limit ≤ 200`).
- Obligation evaluation is lazy + debounced (30s) on reads and relevant writes – no cron to deploy or forget.

Surface today: work orders (list/detail/status/feed/comments/messages), statuses, KPIs, activity, principals, quotes (CRUD + submit/approve/send/reject), payment requests, pulse/obligations/notifications/snooze. ~30 endpoints across 8 route files.

8 DEVELOPMENT SOP – HOW WE BUILD

This section is new – it codifies the process already practiced in Sprints 1–5 so it survives contact with more builders, contractors, or AI agents. Cadence: **1-week sprints, heavily AI-assisted, every sprint ends demoable**. Capacity assumption: one full-time builder + AI agents; resize sprints if that changes.

8.1 • The sprint loop

Step	What happens	Artifact
1 • Spec	Write the sprint spec before code, using <code>app/docs/SPRINT1-SPEC.md</code> as the template: scope guardrails + a single exit criterion, DDL for any new tables, the API contract endpoint-by-endpoint, the component list, and an acceptance checklist.	<code>app/docs/SPRINT<n>-SPEC.md</code>
2 • Decompose	Split the sprint into task cards with disjoint file ownership (the S1 pattern ran three parallel build agents with hard file-boundary rules). Every card carries its own verification command.	Task cards in the spec
3 • Build	Implement to the spec. New tables = a new numbered migration (append-only, never edit shipped SQL). Seed updates must stay idempotent. Follow the engineering standards (8.3).	Code + migration + seed
4 • Verify	Run the acceptance checklist top to bottom against seeded data; walk the demo script on hero WO-39403; run self-tests (e.g. business-time arithmetic).	Checked checklist
5 • Demo & gate	Demo to the release-gate audience (§9). Capture feedback and decisions; feed the next spec.	Feedback pack
6 • Commit & record	Commit the sprint (see 8.4), update docs: new decisions into the relevant product doc, new questions into its open-questions list.	Tagged commit + doc updates

8.2 • Ground truth & precedence

- **When documents conflict**, `platform-brainstorm/platform-build-plan-handoff.md` governs – its §0 realignment supersedes everything below it and defines the precedence order between documents.
- **Process truth**: `product/wo-lifecycle.md` (v3). Parity with it comes before improving on it.
- **Scope truth**: `product/feature-roadmap.md` – feature tags [P]/[1]/[2]/[3] decide what a sprint may include.
- **Decisions are recorded, not remembered**. Each product doc carries dated decisions and an open-questions list; specs cite them. Continue the pattern (a lightweight `product/decision-log.md` is recommended once multiple people are deciding).

8.3 • Engineering standards (as practiced – now policy)

- TypeScript ESM everywhere; `tsx` runs server code directly – no build step until deployment demands one.
- Zod validation at every API boundary; the single error envelope; role gates return 403 and the UI renders the identical gate as a locked control with a tooltip – **never hidden**.
- `@theone/shared` stays types-only; the web app never imports the DB runtime.
- Migrations: raw SQL, numbered, append-only. Seeds: idempotent (truncate-and-reload). Demo states are computed with real business-time arithmetic so they land on exact obligation tiers.

- Derived money is computed, never stored (`qty × rate × OT`). Status groups are the one sanctioned denormalisation.
- Every write is attributed in `activity_log` — humans and service accounts alike.
- Plain CSS design tokens; the `UI Essentials/` brand kit is authoritative (Seamless blue #35b8f3, navy #0a1628, Barlow). The HTML mockups are layout references only.
- Don't bake "The One" into identifiers — the name is a placeholder.

8.4 • Version control (tightened – current practice is the gap)

Today the entire repo is one commit, with Sprint 5 sitting uncommitted, in a OneDrive-synced folder with no remote. One sync conflict or bad edit and there is no restore point. The three changes below cost minutes and remove the whole risk class.

1. **Commit at least per sprint** — message pattern `S<n>: <scope> – <exit criterion met>`. Commit the current S5 work now.
2. **Add a private remote** (GitHub) and push after every commit. Git inside OneDrive is a known corruption/conflict hazard; the remote is the real backup. (Optionally move the working copy out of OneDrive and let GitHub be the sync.)
3. **Tag release gates** (`r0-prototype` , `r1-pilot` , ...). Branch per sprint (`sprint/s6-vendors`) once more than one person or agent commits; PRs become mandatory at that same moment.
4. **Never commit secrets.** `.env` stays ignored; audit before the first push (see §11 — a sibling repo has live credentials tracked in git).

8.5 • Environments

Stage	Runtime	Promotion trigger
Dev (now)	Local: PGLite + Vite + Fastify (<code>127.0.0.1</code>)	—
Pilot (R1)	Docker-compose Postgres 16 (committed, unused today); LAN or small host; real auth replaces the actor switcher	~5 pilot users need shared state
Production (R2)	Hosted Postgres + API + web; backups; PBI read-replica	Cutover: the 21K-W0 import (S17) and rollout waves (S18)

8.6 • House rules (non-negotiable)

- ClickUp is READ-ONLY, forever, from every project. GET only — no writes to the live workspace under any circumstances.
- The `client_visible` boundary is schema-critical. Nothing reaches a client surface without it; the Quo channel is never client-facing by construction.
- **Obligations have no dismiss.** Evidence resolves them; snoozing requires a reason. Do not add a dismiss endpoint "temporarily".
- **Parity before improvement.** Every lifecycle stage in §4 must work at parity before the stage is redesigned.

9 QA, ACCEPTANCE & RELEASE GATES

Per-sprint quality bar

- 1. **Acceptance checklist** – written in the spec before build, executed top-to-bottom against seeded data before the sprint demo. A sprint without a fully checked list does not close.
- 2. **Scripted UAT on hero WO-39403** – the seeded 7-Eleven refrigeration WO walks all 15 stages (the S4 demo script). New features must appear in that walk wherever the lifecycle touches them.
- 3. **Role-gate tests** – exercise both sides of every gate (e.g. an `om` actor must see locked controls and receive 403s; `senior_om` / `atl` must pass). Seeded personas exist for exactly this.
- 4. **Deterministic demo states** – seed anchors are computed with the real business-time arithmetic so obligation tiers land exactly; keep them that way (no hand-set timestamps).
- 5. **Self-tests** – pure logic (business-time, money math, audit rules) carries its own test file, runnable via `tsx`. Grow this set with every rule added.
- 6. **Migration + seed drill** – from an empty database: migrate → seed → smoke (health, WO list, WO detail, quote, pulse). Also verify migrate-without-seed (endpoints must degrade gracefully, as the obligation routes already do).
- 7. **UX audit rules** (§6) apply to every new screen: labels, required markers, disabled styling, contrast, locked-not-hidden, tabular numerals.

Release gates

Release	Exit gate – verbatim commitments
R0 · Prototype (S1–S4)	Leadership + one each of OA / OM / AR confirm "this is our process." Feedback pack goes to the builder. Status: reached.
R1 · Pilot (S5–S10)	Dispatch runs "Incoming WOs" intake for one week with zero ClickUp fallback (~5 users live).
R2 · Cutover (S11–S18)	ClickUp Vista space goes read-only, then decommissioned. Requires: parity + money + sync + the 21K-WO import with count reconciliation.
R3 · Differentiators (S19+)	Scoped per feature (vendor portal, AI triage, PM/assets, absorptions); no single gate.

Recommended addition at R1: a written **pilot incident path** – who a pilot user pings when something breaks, where bugs are logged (GitHub issues once the remote exists), and the rollback rule (pilot reverts to ClickUp for that WO, never edits the DB by hand).

10 RELEASE PLAN & CURRENT POSITION

Source of truth: `product/feature-roadmap.md` release plan. Cadence: 1-week sprints.

Sprint	Scope	Status
S1-S4	R0 Prototype: skeleton + schema + brand shell; WO record + statuses + activity + <code>client_visible</code> ; intake→hire flows + stubs; demo dashboard + scripted 15-stage demo	Done
S5	Obligation engine + Pulse + Receivables + notifications (built); auth + RBAC v1 (outstanding — the S5 roadmap line)	In progress, uncommitted
S6	Vendor DB v1: trades, geo, rates, pooled map + my book	Next
S7	Quote builder v1 hardening, NTE enforcement, soft-close checklists w/ photo upload	—
S8	Ecotrak live intake + REST API v1 + webhooks (n8n connected)	—
S9	Saved views, role dashboards v1, SLA aging	—
S10	Hardening; pilot go-live	—
S11-S18	R2: automations + escalations · compliance module · invoicing + QuickBooks · outbound sync + email-to-WO · map/exec/PBI · ClickUp import (21K) · rollout waves + Teams retirement	—
S19+	R3: vendor portal + auto-assign escalation · VoIP · AI triage + copilot · client portal · scorecards + 1099 · PM/assets · Argus absorption · Seamgo decision	—

Immediate next actions

1. Commit Sprint 5; add the GitHub remote; tag `r0-prototype` at the S4 point (8.4).
2. Write `SPRINT5-SPEC.md` retroactively-lite (scope + acceptance checklist as-built) or fold it into `SPRINT6-SPEC.md` ; resume spec-first from S6.
3. Close the S5 roadmap line: auth + RBAC v1 to replace the actor switcher before pilot users touch the app.
4. Run the Corrigo/ServiceChannel API feasibility spike — R2's outbound-sync scope hangs on it.
5. Answer the top process questions blocking build decisions (§11): decline recording, on-site approval recording, tech payment terms/approval.

11 RISKS, OPEN DECISIONS & SECURITY NOTES

Standing risks

Risk	Mitigation / owner
Corrigo + ServiceChannel API feasibility unproven	Spike ASAP; R2 outbound-sync scope depends on it. Ecotrak (staging creds in hand) goes first.
Seamgo & Argus boundaries	Absorb-vs-bridge decisions owed by R2; module boundaries (intake, dispatch/vendors, quoting, sync, AR, invoicing) are drawn now so absorption is a merge, not a rewrite.
Per-person ClickUp lists = OM books?	Confirm before S6 routing hardening and the S17 import mapping.
Build capacity = 1 builder + AI	Sprints are sized for this; resize the plan if capacity changes.
Single-commit repo in OneDrive, no remote	§8.4 – commit, push, tag. Highest-leverage 30 minutes available.
Facilio as fallback	Evaluation checklist ready; portfolio pricing from ~\$25K/yr. Keep as leverage.

Open process questions (from the lifecycle doc – answers unblock build)

1. Accept/decline criteria – and where is a decline recorded today?
2. Confirm TL/ATL Audit and TL Rep placement (between quote and approval?).
3. Where does an on-site approval get recorded, and who confirms it with the client?
4. Do emergency WOs compress/skip stages (dispatch before quote)?
5. Who performs collection calls – AR team or a separate role?
6. Per-CMMS differences in outbound sync at TL Rep?
7. Tech payments: per visit or on completion? Are assessment-only visits paid? Terms? Who approves the payable?
8. Does AP also settle parts/materials invoices?
9. Confirm the lifecycle-stage → status mapping table (§4).

Security notes

Finding	Action
Quotato repo tracks <code>.mcp.json</code> with live credentials in git	Rotate those credentials and purge the file from history before any absorption into this monorepo or any push to a shared remote.
Quotato webhooks lack replay protection	Add timestamp + signature-window checks when the integration is consumed as a sibling service (the recommended path).
Quotato uses <code>prisma db push</code> – no migration history	Snapshot its schema before integrating; The One's append-only SQL migration discipline is the standard.
Pre-auth prototype (X-Actor-Id header)	Fine locally; must be replaced by real auth + RBAC before any network-exposed pilot (S5 roadmap line).
ClickUp API token	Lives outside the repo (correct); keep it out, and keep all ClickUp access GET-only per the house rule.

12 DOCUMENT MAP

This pack summarizes; these files govern. Paths are relative to the repo root.

Document	Role
platform-brainstorm/platform-build-plan-handoff.md	Governing handoff brief. §0 realignment supersedes all older docs; defines precedence.
product/wo-lifecycle.md (v3)	Process source of truth – 15 stages, roles, status mapping, open questions.
product/feature-roadmap.md (v1)	Scope source of truth – feature map L0–L11 with release tags; sprint plan R0–R3; standing risks.
product/quotes-payments.md	Requirements: quote builder, tech payments, UX corrections, decided answers.
product/quo-map.md	Reverse-engineering map of Quotato + integration options and risks.
platform-brainstorm/PLAN.md	Stakeholder master plan: 11 working decisions, phases, risks, next actions.
platform-brainstorm/platform-architecture-proposal.md	Architecture ADR: three scored candidates; sequence recommendation; 8 open decisions.
platform-brainstorm/clickup-workspace-map.md · clickup-map-handoff.md	Measured as-is inventory of the live workspace (98 users, 163 lists, 19 statuses, 102 fields) + portable API handoff. ClickUp = read-only, forever.
platform-brainstorm/clickup-feature-inventory.md	Tiered T0–T3 requirements catalogue of ClickUp's own feature set.
app/docs/SPRINT1-SPEC.md	Sprint-spec template: guardrails, DDL, API contract, acceptance checklist, task cards.
Support Automation/mvp plan.md + future full system plan.md	Argus (completion audit) plans – audit state machine, rules engine, graduation criteria (≥98% agreement).
Inbox Monitor/ESCALATION-BOT.md	Escalation/SLA rulebook – the direct source of the obligation engine's rules.
mockup/design/SPEC.md · UI Essentials/	Mockup vocabulary (layout reference only) · authoritative brand kit.
This pack	The packaged layer: personas, journeys, dev SOP, QA/release process, risk register, index. Update it at each release gate.

Prepared 2026-08-07 with Claude Code from the repository's primary documents and the as-built prototype. Working name "The One" is a placeholder – do not bake it into identifiers.